

Application de gestion des tâches

AUTEUR
Djibril DIASSE

Sommaire

I	Présentation du projet
II	Base de données
III	Backend (NestJS)
IV	Frontend (Angular)
V	Démarche de travail
—	Conclusion générale

PARTIE I

Présentation du projet

1. Présentation générale

Le projet consiste à développer une application web de gestion des tâches permettant d'organiser, d'assigner, de suivre et de valider les tâches au sein d'une équipe de travail. L'objectif principal est d'améliorer la collaboration entre les différents utilisateurs grâce à une gestion des rôles, un suivi des tâches et un système de commentaires favorisant les échanges.

L'application repose sur une architecture sécurisée dans laquelle chaque utilisateur dispose uniquement des fonctionnalités correspondant à son rôle. Le processus de traitement d'une tâche suit un cycle de validation permettant d'assurer un contrôle efficace des travaux réalisés par les collaborateurs.

✓ En bref

- Trois rôles distincts : administrateur, manager, collaborateur.
- Cycle de validation complet des tâches, du brouillon à la validation.
- Sécurité assurée par une authentification JWT + Refresh Token.

2. Objectifs du projet

- ▶ Centraliser la gestion des tâches.
- ▶ Faciliter l'attribution des tâches aux collaborateurs.
- ▶ Assurer le suivi de l'avancement des tâches.
- ▶ Permettre la validation ou le refus des travaux réalisés.
- ▶ Favoriser la communication entre les utilisateurs grâce aux commentaires.
- ▶ Garantir la sécurité des accès grâce à une authentification basée sur JWT.

3. Rôles des utilisateurs

Administrateur

L'administrateur est responsable de la gestion des utilisateurs. Il peut créer un administrateur, créer un manager, créer un collaborateur, consulter la liste de tous les utilisateurs et consulter la liste de toutes les tâches. L'administrateur n'intervient pas dans le traitement ou la validation des tâches.

Manager

Le manager supervise le travail des collaborateurs. Il peut créer une tâche, l'assigner à un collaborateur, consulter les tâches, ajouter des commentaires, valider une tâche, refuser une tâche ou la remettre en brouillon. Une tâche ne peut être validée, refusée ou remise en brouillon que lorsqu'elle a été préalablement soumise par le collaborateur.

Collaborateur

Le collaborateur est chargé de réaliser les tâches qui lui sont attribuées. Il peut consulter ses tâches, soumettre une tâche pour validation et ajouter des commentaires.

4. Cycle de vie d'une tâche

- ▶ Brouillon : tâche créée mais non soumise.
- ▶ Soumise : tâche envoyée au manager pour évaluation.
- ▶ Validée : tâche acceptée par le manager.

- ▶ Refusée : tâche rejetée par le manager.
- ▶ Retour en brouillon : la tâche nécessite des modifications avant une nouvelle soumission.

5. Fonctionnalités principales

- ▶ Authentification sécurisée des utilisateurs.
- ▶ Gestion des utilisateurs.
- ▶ Gestion des tâches.
- ▶ Attribution des tâches.
- ▶ Validation des tâches.
- ▶ Gestion des commentaires.
- ▶ Gestion des rôles et des permissions.
- ▶ Gestion des sessions avec Refresh Token.

6. Technologies utilisées

Technologie	Rôle
NestJS	Développement de l'API REST côté serveur
Angular	Développement de l'interface utilisateur (SPA)
Prisma ORM	Manipulation et migration de la base de données
PostgreSQL	Système de gestion de base de données relationnelle
JWT	Authentification et identification des utilisateurs
Refresh Token	Renouvellement sécurisé des sessions
Docker	Conteneurisation du backend et du frontend
TypeScript	Langage de développement typé, backend et frontend
Tailwind CSS	Mise en forme de l'interface Angular

7. Architecture générale

Le projet adopte une architecture modulaire où chaque fonctionnalité est indépendante (authentification, utilisateurs, tâches, commentaires, etc.). Cette organisation facilite la maintenance, les évolutions futures et la réutilisation du code, tout en respectant les bonnes pratiques de développement avec NestJS.

PARTIE II

Base de données

1. Présentation générale

La base de données constitue le cœur de l'application de gestion des tâches. Elle permet de stocker de manière structurée l'ensemble des informations nécessaires au fonctionnement du système, notamment les utilisateurs, les tâches, les commentaires et les jetons de rafraîchissement utilisés pour l'authentification.

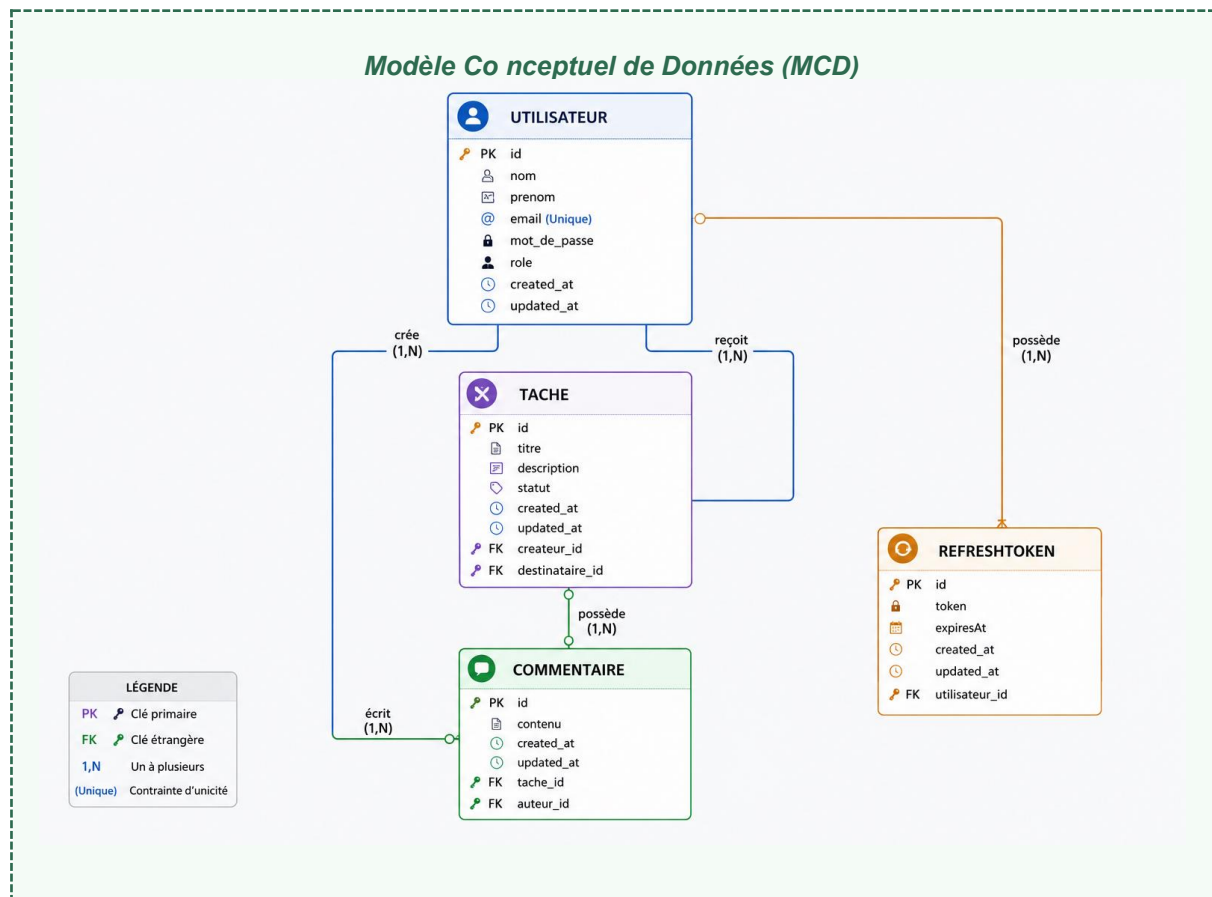
Le modèle de données a été conçu afin d'assurer l'intégrité des informations, de limiter les redondances et de faciliter les échanges entre les différentes fonctionnalités de l'application. Une base de données relationnelle a été retenue afin de garantir la cohérence des données grâce aux clés primaires, aux clés étrangères et aux contraintes d'intégrité.

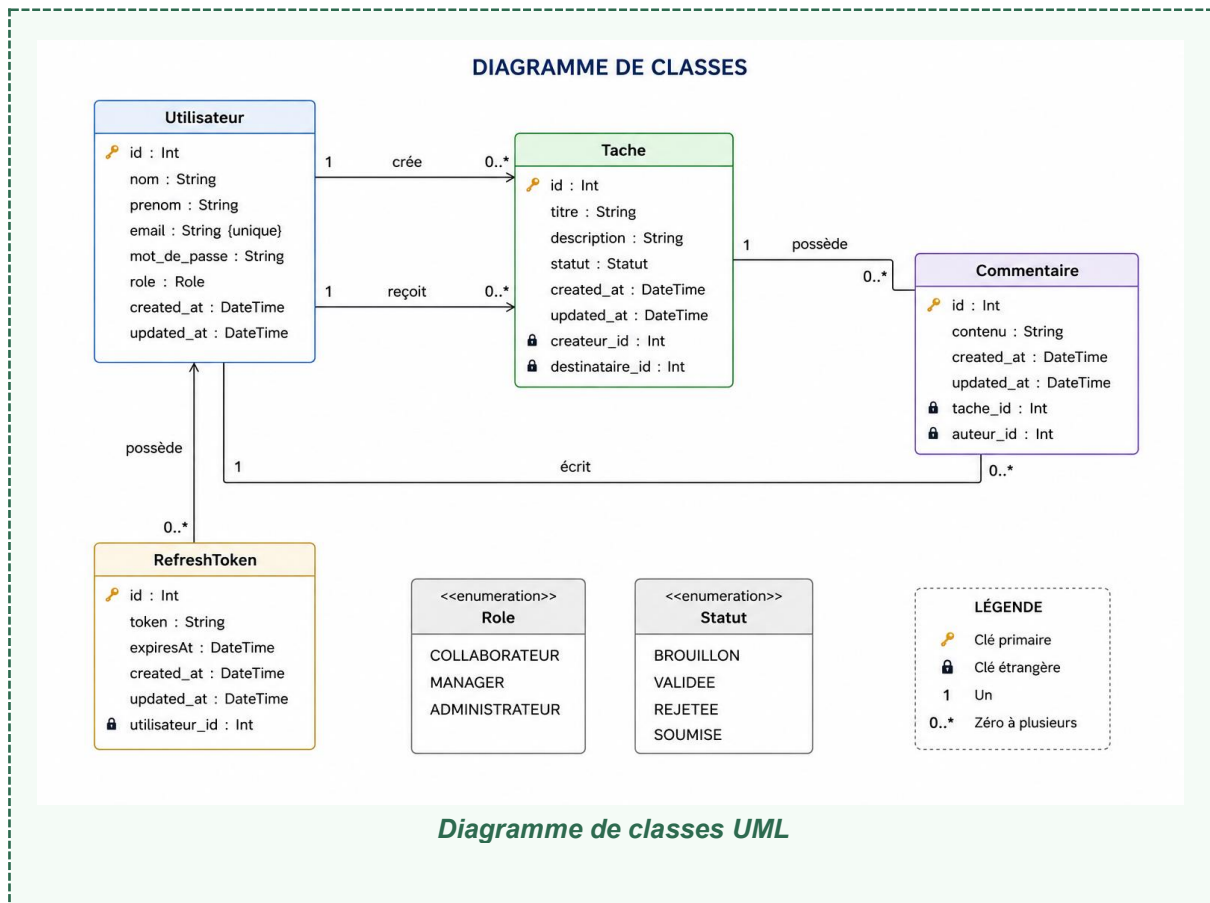
2. Système de gestion de base de données

L'application utilise PostgreSQL comme système de gestion de base de données relationnelle (SGBDR). PostgreSQL a été choisi pour sa robustesse et sa stabilité, ses bonnes performances, sa prise en charge des relations complexes, le respect des propriétés ACID, sa compatibilité avec Prisma ORM ainsi que sa facilité de maintenance et d'évolution.

3. Modélisation de la base de données

La conception de la base de données a été réalisée à partir d'un Modèle Conceptuel de Données (MCD), puis traduite en schéma relationnel avant son implémentation avec Prisma ORM. Cette modélisation permet de définir les entités de l'application, leurs attributs, les relations entre elles, les cardinalités ainsi que les contraintes d'intégrité.





4. Description des principales tables

Table utilisateurs

Cette table contient les informations relatives aux utilisateurs de l'application : identifiant, nom, prénom, adresse e-mail, mot de passe, rôle, dates de création et de mise à jour.

Table tâches

Cette table enregistre toutes les tâches créées dans le système. Chaque tâche possède un créateur, un destinataire, un titre, une description, un statut ainsi que les dates de création et de modification.

Table commentaires

Cette table stocke les commentaires associés aux différentes tâches. Chaque commentaire est lié à une tâche et à son auteur, ce qui permet d'assurer le suivi des échanges entre managers et collaborateurs.

Table refresh_tokens

Cette table conserve les jetons de rafraîchissement utilisés lors de l'authentification. Elle permet de prolonger une session utilisateur sans nouvelle authentification, d'améliorer la sécurité de l'application et de gérer la déconnexion des utilisateurs.

5. Relations entre les tables

Les différentes tables sont reliées entre elles par des clés étrangères. Les principales relations sont les suivantes :

- ▶ Un utilisateur peut créer plusieurs tâches.
- ▶ Un utilisateur peut recevoir plusieurs tâches.
- ▶ Une tâche peut contenir plusieurs commentaires.
- ▶ Un utilisateur peut écrire plusieurs commentaires.
- ▶ Un utilisateur peut posséder plusieurs Refresh Tokens.

Ces relations garantissent la cohérence des données et assurent un fonctionnement correct de l'application.

PARTIE III

Backend (NestJS)

1. Présentation générale

Le backend de l'application a été développé avec NestJS, un framework Node.js basé sur TypeScript permettant de concevoir des applications serveur modernes, évolutives et facilement maintenables. Il expose une API REST permettant au frontend Angular de communiquer avec le serveur afin de gérer l'authentification, les utilisateurs, les tâches, les commentaires et les sessions utilisateurs.

2. Architecture du backend

Le backend adopte une architecture modulaire basée sur les bonnes pratiques de NestJS. Les principaux modules développés sont Auth (authentification, tokens, sessions), Utilisateurs (création et gestion des comptes), Tâches (création, attribution, suivi, validation), Commentaires (échanges entre utilisateurs) et Prisma (communication avec la base de données).

Chaque module s'organise autour de Controllers, qui reçoivent les requêtes HTTP et appellent les services correspondants sans contenir de logique métier ; de Services, qui contiennent la logique métier (création et modification des tâches, gestion des utilisateurs, validation des permissions, interactions avec la base de données) ; et de DTO (Data Transfer Objects), qui définissent et valident la structure des données échangées avec le frontend, garantissant un typage précis et évitant l'envoi de données incorrectes ou inutiles.

```

backend/
├── src/
│   ├── auth/ login, register, refresh, JWT
│   │   ├── decorator/ current-user, public
│   │   ├── dto/
│   │   ├── guard/ jwt-auth.guard
│   │   ├── strategies/ jwt.strategy
│   │   ├── auth.controller.ts
│   │   ├── auth.module.ts
│   │   └── auth.service.ts
│   ├── taches/ création, statuts, assignation
│   │   ├── dto/
│   │   ├── guard/ create, findAll, findOne, update-delete...
│   │   ├── taches.controller.ts
│   │   ├── taches.module.ts
│   │   └── taches.service.ts
│   ├── commentaires/ fil de commentaires par tâche
│   │   ├── dto/
│   │   ├── guard/
│   │   ├── commentaires.controller.ts
│   │   ├── commentaires.module.ts
│   │   └── commentaires.service.ts
│   ├── utilisateurs/ gestion des comptes
│   │   ├── dto/
│   │   ├── guard/ admin.guard
│   │   ├── utilisateurs.controller.ts
│   │   ├── utilisateurs.module.ts
│   │   └── utilisateurs.service.ts
│   ├── prisma/ connexion à la base
│   │   ├── prisma.module.ts
│   │   └── prisma.service.ts
│   ├── common/services/ utilitaires partagés
│   ├── config/ jwt.config
│   ├── app.module.ts
│   └── main.ts
├── prisma/ définition du schéma de données
│   ├── migrations/
│   │   ├── schema.prisma
│   │   └── seed.ts comptes de démonstration
└── Dockerfile

```

Architecture du backend

3. Typage des données et communication

L'ensemble du backend est développé en TypeScript, ce qui permet de définir la structure des objets manipulés, d'éviter les erreurs liées aux mauvais formats de données et d'améliorer la lisibilité du code. Prisma ORM génère automatiquement des types correspondant aux modèles PostgreSQL, garantissant une cohérence entre la base de données, le backend et le frontend.

La communication entre Angular et NestJS se fait via une API REST échangeant des données au format JSON : le frontend envoie une requête HTTP, le contrôleur la reçoit, les données sont validées par les DTO, le service exécute la logique métier, puis le backend retourne une réponse JSON.

4. Authentification et sécurité

- Authentification par JWT : un Access Token est généré pour accéder aux ressources protégées, un Refresh Token permet de renouveler la session sans nouvelle connexion.

- ▶ Hachage des mots de passe : les mots de passe ne sont jamais enregistrés en clair, ils sont transformés par un algorithme de hachage avant stockage.
- ▶ Hachage des Refresh Tokens : ils sont également hachés avant insertion en base afin d'empêcher toute utilisation frauduleuse en cas de fuite de données.
- ▶ Guards NestJS : l'Auth Guard JWT vérifie la validité du token et identifie l'utilisateur connecté, tandis que les Guards de rôles limitent l'accès selon le profil (Administrateur, Manager, Collaborateur).
- ▶ CORS : configuré côté backend pour autoriser explicitement le frontend Angular à communiquer avec l'API.

✓ Sécurité en un coup d'œil

- Mots de passe et Refresh Tokens hachés avant stockage.
- Accès aux routes filtré par rôle grâce aux Guards.
- CORS restreint aux origines autorisées.

5. Avantages de l'architecture adoptée

Cette architecture offre une organisation claire du code, une séparation nette entre contrôleurs, services et logique métier, un typage fort grâce à TypeScript, une communication standardisée en JSON, une sécurité renforcée grâce aux JWT et aux Guards, ainsi qu'une intégration efficace avec PostgreSQL via Prisma ORM.

PARTIE IV

Frontend (Angular)

1. Présentation générale

Le frontend de l'application a été développé avec Angular, un framework TypeScript permettant de créer des interfaces web modernes, dynamiques et organisées. Il constitue la partie visible de l'application et communique avec le backend NestJS à travers une API REST en échangeant des données au format JSON.

2. Architecture du frontend

Le frontend repose sur l'architecture modulaire d'Angular. Chaque page possède son propre composant, composé d'un fichier TypeScript (logique, gestion des données, appels aux services, événements utilisateur), d'un fichier HTML (structure visuelle, formulaires, tableaux, boutons) et de styles (mise en forme réalisée avec Tailwind CSS). Angular utilise le data binding pour synchroniser automatiquement les données entre la logique et l'interface.

Architecture du frontend

```
frontend/
├── src/
│   ├── app/
│   │   ├── guard/  authentication et rôles par route
│   │   ├── interceptors/  auth.interceptor (JWT + refresh)
│   │   ├── pages/
│   │   │   ├── auth/  login, register
│   │   │   ├── admin/  dashboard, utilisateurs, création de compte
│   │   │   ├── manager/  collaborateurs, popup d'assignation
│   │   │   ├── manager-collaborateur/  liste de tâches partagée
│   │   │   ├── tout/  page tâche + commentaires (commune)
│   │   ├── services/  api, auth, tâche, commentaire, utilisateur
│   │   ├── types/  interfaces TypeScript
│   │   ├── utilitaire/sidebar/  navigation adaptative par rôle
│   │   ├── app.config.ts
│   │   ├── app.routes.ts
│   │   └── app.ts
│   ├── environments/  configuration API par environnement
│   ├── index.html
│   ├── main.ts
│   └── Dockerfile
```

3. Typage des données

Comme le backend, le frontend utilise TypeScript afin d'assurer un meilleur contrôle des données manipulées. Des interfaces et des types ont été créés pour représenter les entités de l'application, notamment les utilisateurs (identifiant, nom, prénom, email, rôle) et les tâches (identifiant, titre,

description, statut, créateur, destinataire), ce qui facilite la communication avec l'API et assure la cohérence entre le frontend et le backend.

4. Services Angular

Les services Angular centralisent la communication avec le backend via le module HttpClient (GET, POST, PUT/PATCH, DELETE). Chaque fonctionnalité possède son service dédié : AuthService (connexion, déconnexion, renouvellement du token), UtilisateurService (gestion des utilisateurs), TacheService (création, récupération, modification des tâches) et CommentaireService (gestion des commentaires).

5. Routes et sécurité

La navigation est gérée par l'Angular Router, avec des routes dédiées telles que /login, /dashboard, /taches, /utilisateurs et /profil. Des Guards Angular protègent l'accès aux pages en vérifiant l'authentification, la validité du token JWT et le rôle de l'utilisateur : un administrateur accède à la gestion des utilisateurs, un manager aux fonctionnalités de création et de validation des tâches, un collaborateur uniquement à la consultation et à la soumission de ses tâches.

L'authentification côté frontend repose sur le même mécanisme JWT que le backend : lors de la connexion, Angular envoie les identifiants, reçoit un Access Token et un Refresh Token, conserve les informations de session, puis un mécanisme de renouvellement maintient la session sans nouvelle connexion.

6. Organisation des pages

- ▶ Page d'authentification : connexion à l'application.
- ▶ Tableau de bord : informations principales selon le rôle.
- ▶ Gestion des tâches : consultation, création, attribution, modification du statut.
- ▶ Gestion des utilisateurs : réservée à l'administrateur.
- ▶ Commentaires : échanges autour des tâches.

7. Avantages de l'architecture frontend

L'utilisation d'Angular apporte une organisation claire grâce aux composants, une réutilisation des éléments d'interface, un typage fort avec TypeScript, une séparation entre affichage et logique métier, une sécurité renforcée grâce aux Guards et une interface moderne grâce à Tailwind CSS.

PARTIE V

Démarche de travail

1. Étapes de développement

Le développement du projet a suivi une progression logique, du backend vers le frontend, en passant par la conteneurisation et la sécurisation des échanges :

- ▶ Création du projet backend (initialisation NestJS).
- ▶ Création du projet frontend (initialisation Angular).
- ▶ Mise en place du fichier docker-compose.yml pour orchestrer les services de l'application.
- ▶ Mise en place des deux Dockerfiles, l'un pour le backend et l'autre pour le frontend.
- ▶ Réalisation des migrations de la base de données avec Prisma, reprises et ajustées jusqu'à obtenir un schéma stable et fonctionnel.
- ▶ Implémentation du backend : logique métier, mise en place des Guards pour la sécurité et le contrôle des accès.
- ▶ Configuration des CORS afin d'autoriser le frontend Angular à communiquer avec l'API backend.
- ▶ Implémentation du frontend : logique applicative, mise en place des routes et des pages.
- ▶ Finalisation et utilisation de l'application dans son ensemble.

2. Difficultés rencontrées

Angular et NestJS étant de nouveaux frameworks pour moi, une phase d'adaptation a été nécessaire au démarrage du projet. Il a fallu un certain temps pour me familiariser avec leur fonctionnement respectif.

Cependant, disposant déjà de bons prérequis en JavaScript et TypeScript je me suis rapidement habitué aux deux frameworks. Une fois leur architecture assimilée, le développement est devenu simple et fluide : l'essentiel de l'apprentissage a porté sur la compréhension de l'architecture propre à chaque technologie plutôt que sur le langage lui-même.

PARTIE —**Conclusion générale**

L'application de gestion des tâches développée dans ce projet répond aux objectifs fixés en offrant un environnement collaboratif structuré et sécurisé. La séparation claire des rôles (administrateur, manager, collaborateur), le cycle de validation des tâches et le système de commentaires favorisent une organisation efficace du travail d'équipe.

Sur le plan technique, l'architecture retenue — backend NestJS modulaire, frontend Angular basé sur des composants, base de données PostgreSQL modélisée via Prisma ORM, conteneurisation Docker et authentification JWT avec Refresh Token — garantit à la fois la robustesse, la sécurité et l'évolutivité de l'application. Le typage fort assuré par TypeScript sur l'ensemble de la chaîne limite les erreurs et facilite la maintenance.

La découverte d'Angular et de NestJS a constitué la principale difficulté du projet, rapidement surmontée grâce à des prérequis solides en JavaScript. Ce projet constitue ainsi une base solide, pouvant être enrichie par de nouvelles fonctionnalités (notifications, statistiques d'avancement, pièces jointes, etc.) sans remettre en cause l'architecture mise en place.